What is Stiffness?

Jaemin Oh

February 28, 2024*

Abstract

We often face the terminology "stiffness". What does it mean? This short note explains stiffness in the context of numerical analysis through three examples.

1 Introduction

About two years ago, I decided to work on Physics-informed neural networks (PINNs) [3]. Tons of papers were already out there. Among them, I found several interesting papers describing "failure modes" of PINNs [2], by analyzing stiffness of the gradient flow [7], and the neural tangent kernel [8].

On the other hand, one of my friends explained implicit neural representations (INRs) to me, currently known as neural fields [4]. It was not difficult to recognize a close connection between PINNs and INRs, in that both methods tried to approximate implicitly defined functions (solutions to differential equations, and synthesized views) with neural networks. However, the approximation qualities were not that great for both fields. In INRs, one of the major approaches to improving the quality was overcoming the spectral bias. For example, it was empirically validated that adding a fixed Fourier feature layer improved the quality [5]. Using the sine activation function was another example [4].

Therefore, I had confused stiffness with the spectral bias. It seemed to me that both concepts were primary sources of poor solution quality of PINNs. Moreover, my PhD advisor had said to me that stiffness means some singular behavior of the solution. So what does stiffness mean?

2 Stiffness

2.1 Ordinary Differential Equations

Stiffness in ordinary differential equations (ODEs) refers to a situation in which explicit methods do not work well [6, Chapter 1.8].

*edit: June 12, 2024

We begin with a typical example describing the stability of the forward Euler method. Let us consider the following simple ODE,

$$y'(t) = -Cy(t), \quad y(0) = y_0,$$

where $t \in [0, \infty)$ and C is a positive real number. The analytic solution is

$$y(t) = y_0 e^{-Ct}.$$

It is not necessary to solve this simple ODE with numerical methods, but by doing so we can get some insights. The simplest numerical method is the forward Euler method,

$$y(t_{i+1}) = y(t_i) - \Delta t C y(t_i) = (1 - \Delta t C) y(t_i),$$

where $t_i = i\Delta t$, for $i \in \mathbb{Z}_{\geq 0}$. The solution to the recurrence relation is

$$y(t_i) = (1 - \Delta t C)^i y(0).$$

If $|1 - \Delta t C| \geq 1$, the solution to the recurrence relation cannot approximate the analytic solution, as it merely oscillates or diverges. The convergence of the discretized solution requires $|1 - \Delta t C| < 1 \Leftrightarrow \Delta t < 2/C$. Hence the larger the coefficient C, the smaller the time step size Δt for explicit methods.

This example essentially describes what is stiffness: the problem requires fine discretization for computation.

2.2 Partial Differential Equations

Stiffness in ODEs discussed in section 2.1 can explain stiffness in partial differential equations (PDEs). It is essentially the same, in that stiff PDEs require fine discretization in time. The fundamental reason is that numerical methods to solve PDEs usually convert PDE to a system of ODEs. To get a grasp, let us begin with a concrete example.

Let's consider the Kuramoto-Sivashinsky equation, which reads

$$\partial_t u + \partial_x^2 u + \partial_x^4 u + u \partial_x u = 0,$$

on a periodic spatial domain $[0, 32\pi)$. Due to the periodicity of the spatial domain, Fourier transform-based methods would be the best choice. Here, we consider the Fourier spectral-collocation method, since it deals with nonlinear terms easily.

Assume that we have discretized the spatial domain by N uniform grid, $x_j = 32\pi(j-1)/N$, for $j = 1, \ldots, N$ and for some even number N. Let $P = 32\pi$ and $k \in \{-N/2 + 1, \ldots, N/2\}$. Then Fast Fourier Transform (FFT) computes

$$\hat{u}_k := \mathcal{F}[u](k) = \sum_j u(x_j) e^{-i\frac{2\pi k}{P} x_j}.$$

We denote $\hat{u} : k \mapsto \hat{u}_k$. The inverse transform computes

$$\tilde{u}(x_j) := \mathcal{F}^{-1}[\hat{u}](x_j) = \frac{1}{N}\sum_k \hat{u}_k e^{i\frac{2\pi k}{P}x_j}.$$

Note that $\tilde{u}(x_j) = u(x_j)$ holds exactly. Thus, $\tilde{u}$ is a trigonometric interpolant of u at $\{x_j\}_{j=1}^N$. Fourier transform converts $\partial_x u$ to the multiplication of $i \cdot \xi$ and corresponding Fourier modes, where $\xi = \frac{2\pi k}{P}$ is the wavenumber. That is,

$$\mathcal{F}[\partial_x u](k) = i\xi\hat{u}_k.$$

The Fourier spectral-collocation method converts the Kuramoto-Sivashinsky equation to the following system of ODEs:

$$\partial_t \hat{u}_k(t) = -(i\xi)^2\hat{u}_k(t) - (i\xi)^4\hat{u}_k(t) - \frac{1}{2}(i\xi)\mathcal{F}\left[\mathcal{F}^{-1}\left[\hat{u}_k(t)\right]^2\right].$$

Stiffness occurs due to $\xi^2 - \xi^4$, which is the coefficient of the linear term $\hat{u}_k(t)$. For example, let's fix $N = 128$. Then, for $k = N/2 = 64$, $\xi^2 - \xi^4 = 4^2 - 4^4 = -240$. Thus, for the forward Euler time discretization, the step size Δt should be less than $2/240$ at least. However, the Kuramoto-Sivashinsky equation exhibits chaotic long-time behavior, which is most interesting. If simulation time $t_{\max}$ is large, say 90, then the forward Euler method requires at least $90 \times 120 = 10800$ steps. Finer time discretization is necessary to compute accurate solutions.

One of the famous time-stepping strategies for stiff PDEs is the exponential time differencing Runge-Kutta method [1]. It overcomes stiffness in the Kuramoto-Sivashinsky equation by solving the linear part exactly, which is the main source of stiffness.

2.3 Gradient Flow

Gradient descent

$$\theta_{i+1} = \theta_i - \eta\nabla_\theta L(\theta_i)$$

is just a forward Euler discretization of the gradient flow

$$\frac{d}{dt}\theta(t) = -\nabla_\theta L(\theta(t)).$$

Therefore, if there are "stiff" components on the right-hand side of the gradient flow, the gradient descent requires an exceedingly small step size, and hence converges slowly.

3 Conclusion

This short note has described the terminology "stiffness" through three examples. It originated from the ODE context, indicating the presence of various time scales. Stiffness in two other contexts, PDEs and gradient flows refers to

essentially the same thing, as their computation requires discretizing a system of ODEs.

Overcoming stiffness is crucial for efficient computation. There are various methods for solving stiff problems and analyzing stiffness. Applying such methods to PINNs would be a promising research direction, unveiling the fundamental reason of why PINNs fail so often and developing a more robust method.

References

[1] A.-K. Kassam and L. N. Trefethen, *Fourth-order time-stepping for stiff pdes*, SIAM Journal on Scientific Computing, 26 (2005), pp. 1214–1233.

[2] A. Krishnapriyan, A. Gholami, S. Zhe, R. Kirby, and M. W. Mahoney, *Characterizing possible failure modes in physics-informed neural networks*, in Advances in Neural Information Processing Systems, M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, eds., vol. 34, Curran Associates, Inc., 2021, pp. 26548–26560.

[3] M. Raissi, P. Perdikaris, and G. E. Karniadakis, *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*, Journal of Computational physics, 378 (2019), pp. 686–707.

[4] V. Sitzmann, J. Martel, A. Bergman, D. Lindell, and G. Wetzstein, *Implicit neural representations with periodic activation functions*, Advances in neural information processing systems, 33 (2020), pp. 7462–7473.

[5] M. Tancik, P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. Barron, and R. Ng, *Fourier features let networks learn high frequency functions in low dimensional domains*, in Advances in Neural Information Processing Systems, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, eds., vol. 33, Curran Associates, Inc., 2020, pp. 7537–7547.

[6] L. N. Trefethen, *Finite difference and spectral methods for ordinary and partial differential equations*, Cornell University-Department of Computer Science and Center for Applied . . . , 1996.

[7] S. Wang, Y. Teng, and P. Perdikaris, *Understanding and mitigating gradient flow pathologies in physics-informed neural networks*, SIAM Journal on Scientific Computing, 43 (2021), pp. A3055–A3081.

[8] S. Wang, X. Yu, and P. Perdikaris, *When and why pinns fail to train: A neural tangent kernel perspective*, Journal of Computational Physics, 449 (2022), p. 110768.